capital

Custom Action Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at:
www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from:
www.mentor.com/terms_conditions/enduser.cfm.

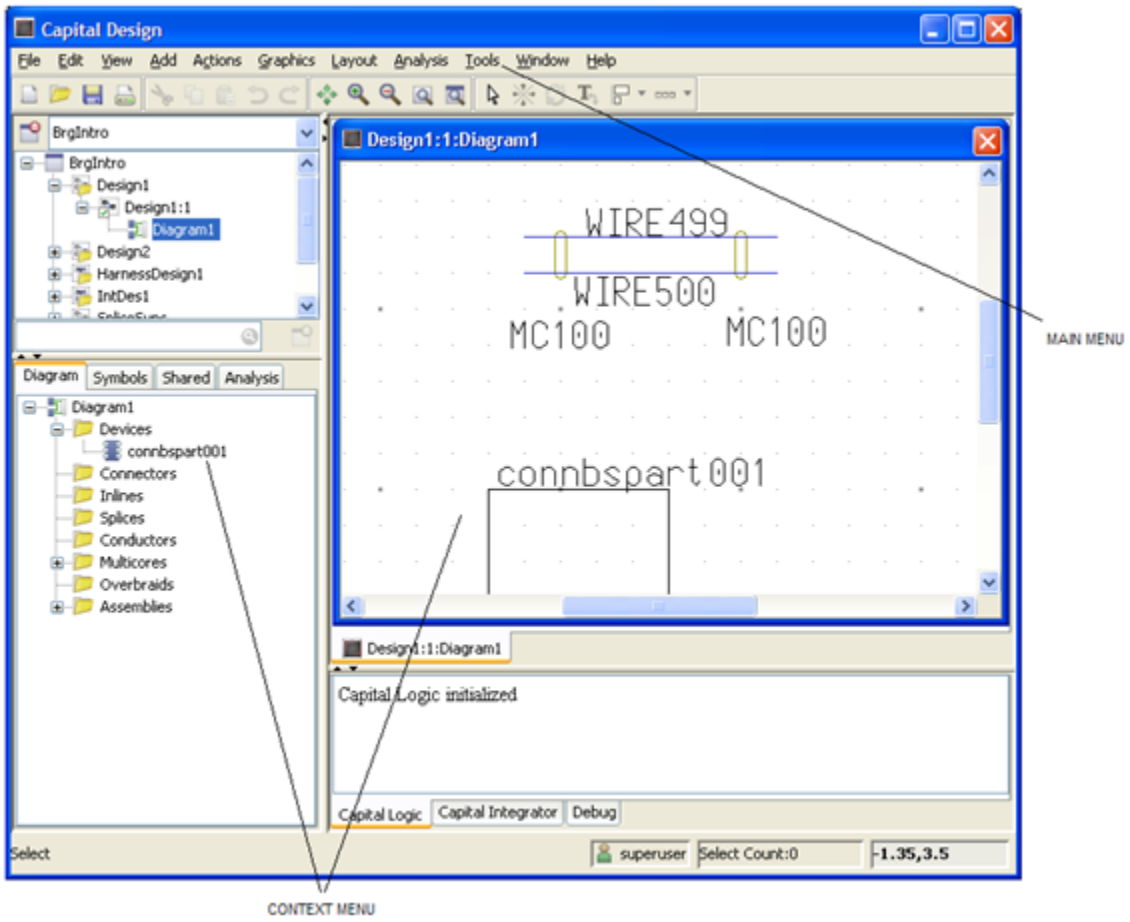
Table Of Contents

Introduction **2**
Interfaces **3**
IXAction 3
IXApplicationAction 5
IXApplicationContext 6
IXOutputWindow 7
Exercise **8**
Guidance **16**
IXApplicationContextListener 16
Displaying Dialogs 16
HTML in the Output Window 17
Changing Properties and Attributes 19
Examples **21**

Introduction

This document describes the process for developing Custom Action plugins for Capital Logic, Capital Topology, Capital Integrator and Capital HarnessXC. This type of plugin allows you to add your own menus that launch your own code. If you are **not** interested in developing Custom Actions, you do not have to read this document. If you are interested in developing Custom Actions, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading and completing the exercises in this document.

The actions can be attached to the main menu of the application and/or the context menu of the application. See picture below:



Your custom actions can display information to the output window or open their own user interfaces. Please note that special types of menu (like check-box menus) are currently not supported.

Interfaces

IXAction

In order to build a Custom Action, this is the interface that you will need to implement:

```
public interface IXAction extends IXPlugin, IXAvailability
{

      enum Trigger
      {
            MainMenu, ContextMenu, InvokeFromPlugin
      }


      Trigger[] getTriggers();

      Icon getSmallIcon();

      Integer getMnemonicKey();

      String getLongDescription();

      boolean isReadOnly();

      boolean execute(IXApplicationContext applicationContext);
}
```

As you can see, `IXAction` extends the `IXPlugin` interface. This means that you will need to implement the `getName()`, `getVersion()` and `getDescription()` methods. Additionally, you will need to implement the following methods:

`getTriggers()` – this must return an array of Trigger values that indicate the menus that this custom action can be used with. If the custom action returns an array with just a { `MainMenu` } entry, then you will only see the custom action menu on the main menu and not the context menu. If the custom action returns an array with { `MainMenu`, `ContextMenu` } entries, then you will see the custom action menu on the main menu and the context menu. An error will be raised if the array returned is null or empty and the custom action will not be available.

`getSmallIcon()` – this can return either null (if no icon for the action is required), or an icon object. If an icon object is returned, a check will be made to ensure the icon has the correct

dimensions: 16 by 16. If this check fails, a warning will be issued in the plugin Tab of the Output Window and the icon will not be used, but the action will still be available.

`getMnemonicKey()` – this can return either null, or an Integer object that represents the key to be used. If not null, a check will be made to ensure that the key is present in the name and that no other custom action has that mnemonic. If these checks fail, a warning will be issued in the plugin Tab of the Output Window and the mnemonic will not be used by the custom action, but the custom action will still be available.

`getLongDescription()`– this will be the long description for the custom action. Whenever the action is active, the string returned by this method will appear in the status bar at the bottom of the application.

`isAvailable( IApplicationContext app )` – (as defined in interface `IXAvailability`) By returning true or false from this method, you can control when the action is available on the custom menu. The application context object passed in will contain information that includes the currently selected objects. If your custom action only works when a backshell is selected, then this method can be used to ensure that the custom action menu is not present if a backshell is not currently selected. For the main menu, this will result in the custom menu entry being faded out. In the context menu, this will result in the custom menu being removed.

`isReadOnly()` – You should use this method to indicate if your custom action needs to modify any properties or attributes on objects. If it does not, then this method should return true. In this case, your custom action will be available in situations where the design you are working on is read-only (locked by someone else) or read-write (locked by you). If you return false from this method, then your action will only ever be available on read-write designs and you will be allowed to modify properties and attributes on objects.

`execute( IApplicationContext app )` – this is the method that will be called when the custom action's menu entry is clicked. This method must return a Boolean that indicates the success or failure of the custom action. This is the main method of your custom action. It is the method that should do the work you need done. It can open a dialog that you create or it may write information to the output window or communicate to an external database or it can send a message to an external service that you develop, etc.

You should not implement `IXAction` directly; you need to implement one of its child interfaces. These child interfaces do not add any additional methods. However, they do indicate the type of design with which your custom action can be used.

To implement an action for Logic designs, you will need to implement the `IXLogicAction`. By implementing `IXLogicAction`, the custom plugin will be available whenever a logic design is open in Capital Design or Capital Logic.

To implement an action for Topology designs, you will need to implement the `IXTopologyAction`. By implementing `IXTopologyAction`, the custom plugin will be available whenever a topology design is open in Capital Logic.

To implement an action for Integrator designs, you will need to implement the `IXIntegratorAction`. By implementing `IXIntegratorAction`, the custom plugin will be available whenever an integrator design is open in Capital Design or Capital Integrator.

To implement an action for Harness designs, you will need to implement the `IXHarnessAction`. By implementing `IXHarnessAction`, the custom plugin will be available whenever a harness design is open in Capital HarnessXC.

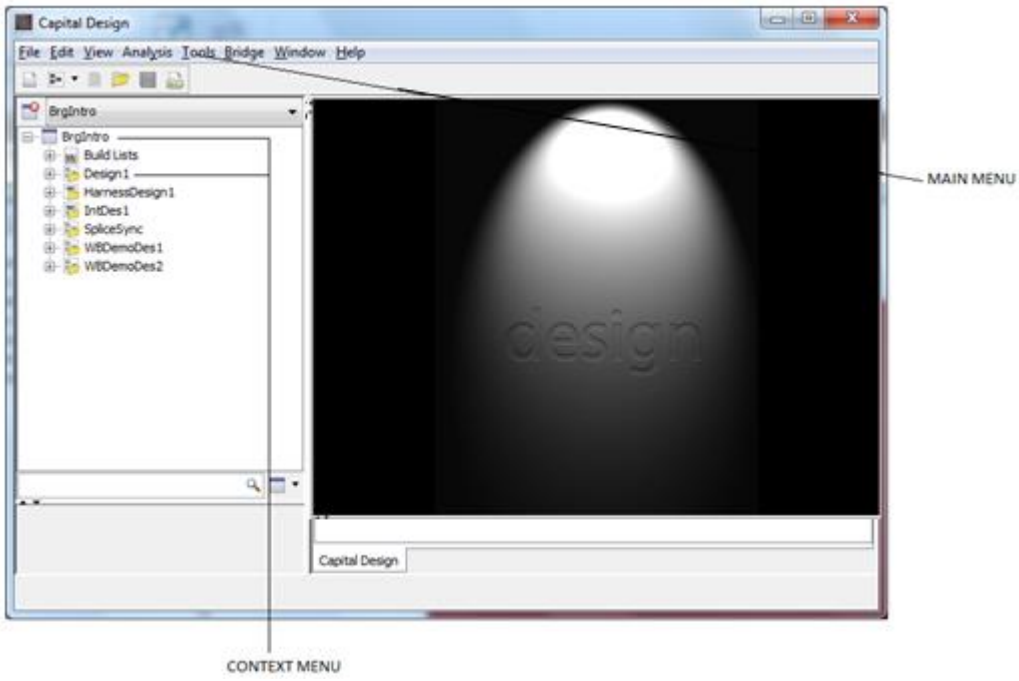
There is nothing stopping you from creating a single custom action that can be used on many different design types. For example, by implementing both the `IXLogicAction` and `IXIntegratorAction` interfaces, the custom plugin you develop will be associated with Capital Logic and Capital Integrator.

These tool based custom actions cannot write to objects outside of the current design in focus. If there is a need to write to objects across designs then such a custom action has to implement the `IXApplicationAction`.

IXApplicationAction

`IXApplicationAction` is a marker interface that extends from `IXAction`. Custom actions that extend `IXApplicationAction` are called custom application actions.

Custom application actions can have write access to objects across designs in the current open project. Thus, such actions have write access to a wider set of objects than the custom actions do. Also note that custom application actions will not be able to work on multiple projects.



This type of actions will be available from the contextual menu in the 'Project browser' and also from the 'Tools' custom main menu. There is no need to open any diagram to be able to use these actions.

IXApplicationContext

This interface allows the `isAvailable()` and `execute()` methods on the `IXAction` interface to make decisions or work on the objects returned by these methods:

`getApplication()` – this returns the current application. This allows a custom action to do slightly different things based on the application being used. The custom dialog that works with Capital Logic and Capital Integrator may require an extra tab in Capital Integrator, for example.

`getCurrentProject()` – this will return the current project the application is working on. Note that an application may have several projects open. This method will return the one that currently has focus.

`getCurrentDesign()` – this will return the current design the application is working on. Note that an application may have several designs open. This method will return the one that currently has focus.

`getSelectedObjects()` – this will return the set of `IXObjects` that are currently selected on the diagram. This method will never return null, but will return an empty set if no objects are selected.

`getLibrary()` – this method will return an `IXLibrary` object. This can be used by customers to search for library parts.

`getOutputWindow()` – returns an `IXOutputWindow` object that allows the custom action to display information to the CAF output window.

IXOutputWindow

This interface allows the custom action to write messages to the output window if one is present. If the output window is not present, then the messages will go to the application log.

`println()` – this method will print the given string to the output window with a newline at the end. You can optionally pass in the name of the tab the message should go to. If no 'tab' name is specified, the output will go to the plugins tab of the output window. If the 'tab' name is specified, a tab for that output will be created (if one does not exist already) and all the output will go to that tab (or cleared from that tab).

`clear()` – this method will clear any text in the output window. If no 'tab' name is specified, the plugins tab will be cleared. If the 'tab' name is specified, the tab with that name will be cleared.

Exercise

In this exercise you will develop a Custom Action using Java. The objective of this custom Action is to display the properties and attributes of the selected objects in the output window.

The exercise is written for the Capital Design application. However, you can use Capital Logic or Capital HarnessXC if you do not have a license for Capital Design.

1) You should start by creating a new plugin from the provided template.

    a. Ensure that your Java development environment has been set-up. If you have completed the Java plugin exercise in the PluginDevelopment.pdf document then it should already be setup. If you have not, then you should stop this exercise and complete it now.

    b. Open a UNIX/DOS console. Ensure that the environment variables required for Java, ANT and Capital have been setup in this console. cd into the following folder within the Capital installation:

    ```
    doc/plugin/examples/Java
    ```

    c. In the above folder, there is a template action. Here is the location of the file as seen from the Capital installation folder:

    ```
    doc/plugin/examples/Java/TemplateConsoleAction.java
    ```

    Create a new folder in the following location within the Capital installation folder:

    ```
    doc/plugin/examples/Java/src/exercise
    ```

    The folder `src` is the directory where all the example Java plugin source code has been placed. This should exist already. The new folder `exercise` is the package in which your plugin will reside.

    d. Copy the template into the new folder you have just created and change its name to `DisplayInfoAction.java`. The result should be a file in the following location:

    ```
    doc/plugin/examples/Java/src/exercise/DisplayInfoAction.java
    ```

    e. Edit this new file.

f. Ensure the plugin is in the correct package. This can be done by adding the following line at the start of the file (before the imports):

```
package exercise;
```

Note: If you use a template as a starting point for your new plugin, you must remember to specify the package it is in.

g. You also need to ensure that the name of the class matches the name of the file. You will need to replace `TemplateConsoleAction` with `DisplayInfoAction` in this file. There are two places where this needs to be done. One is the class definition, the other is the constructor.

h. Change the name returned by the plugin by changing the string returned by the `getName()` method:

```
public String getName()
{
      return "Display Info";
}
```

i. Change the description returned by the plugin by changing the string returned by the `getDescription()` method:

```
public String getDescription()
{
      return "Display Info about the selected component(s).";
}
```

j. Remember to save the file when these changes are completed.

k. Back at the UNIX/DOS console, ensure that you are in the following folder:

```
doc/plugin/examples/Java
```

l. Build your new plugin by typing the following command:

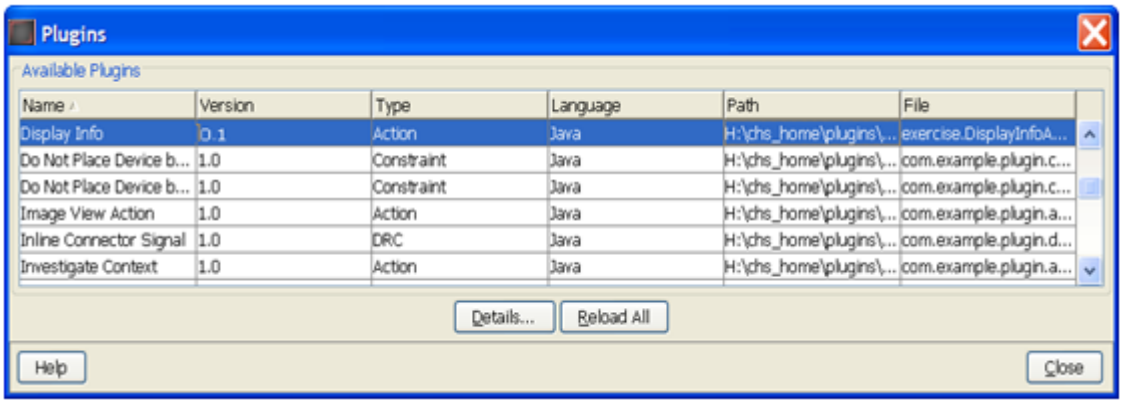
```
ant
```

m. If everything is fine, you will eventually get a “BUILD SUCCESSFUL” message. If you do not, you will need to check the environment variables or check for (and fix) errors in the `DisplayInfoAction.java` file and try again.

n. Start Capital Design (you can use Capital Logic, Capital Integrator or Capital HarnessXC if you wish).

o. From the main menu, click on the File -> Plugins… menu

p. Press the [Reload All] button on the Plugins dialog. You will see your new plugin appear in the list with the other available plugins. See the selected row in the image below:



Leave this dialog open for now.

2) At this point you have a new plugin. However, it has not been modified to work on the currently selected objects. This is your next objective.

a. Edit the `DisplayInfoAction.java` file.

b. Edit the execute() method in the file and add the following code:

```
            public boolean execute(IXApplicationContext
applicationContext)
            {
               // Get the IXOutputWindow so you can log messages.
               IXOutputWindow outputWindow =
                   applicationContext.getOutputWindow();

               // get the set of selected objects
               Set<IXObject> selectionSet =
                   applicationContext.getSelectedObjects();
               for(IXObject selection : selectionSet)
               {
                  outputWindow.println("Name: " + selection.toHTML());

               }
```

```
        // Remember to return true unless something has gone
wrong.
        return true;
    }
```

Note: the **`getSelectedObjects()`** method call. This will return the set of objects that are currently selected on the diagram. For now, we are just displaying the names of the selected objects.

Note: the use of the **`toHTML()`** method call. The will not only insert the name of the object but create a hyperlink that, when clicked will select that object in the application.

c. You will also need to add the following import

```
import java.util.Set;
```

d. Change the version number of the plugin, by editing the getVersion() method:

```
public String getVersion()
{
    return "0.2";
}
```

Note: changing the version number can be important when you are developing a plugin. You can use it to ensure that your plugin has been re-loaded. You can also use it to ensure that everyone using the plugin has the most up-to-date version.

e. Save the file.

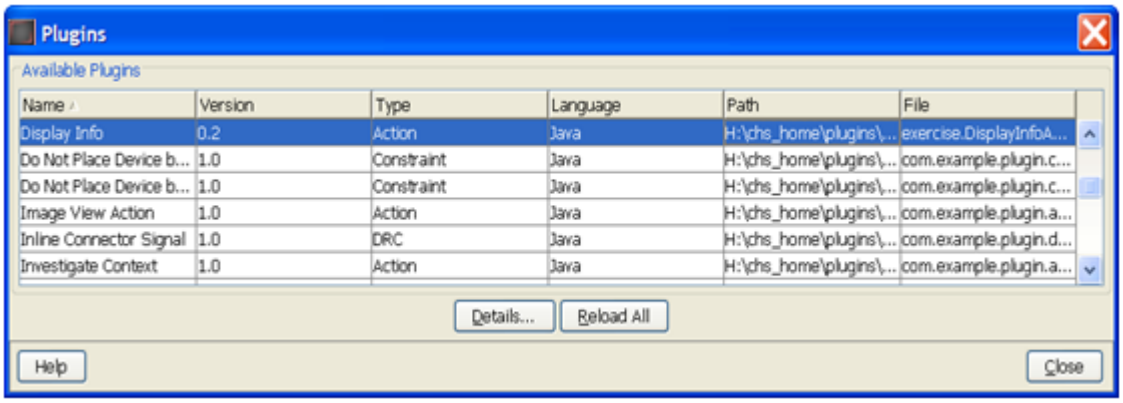
f. Rebuild your plugin by typing the following command in the DOS/UNIX console:

```
ant
```

g. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check for (and fix) errors in the `DisplayInfoAction.java` file and try again.

h. Go back to Capital Design.

i. Press the [Reload All] button on the Plugins dialog. You will see the version number of your new plugin change:



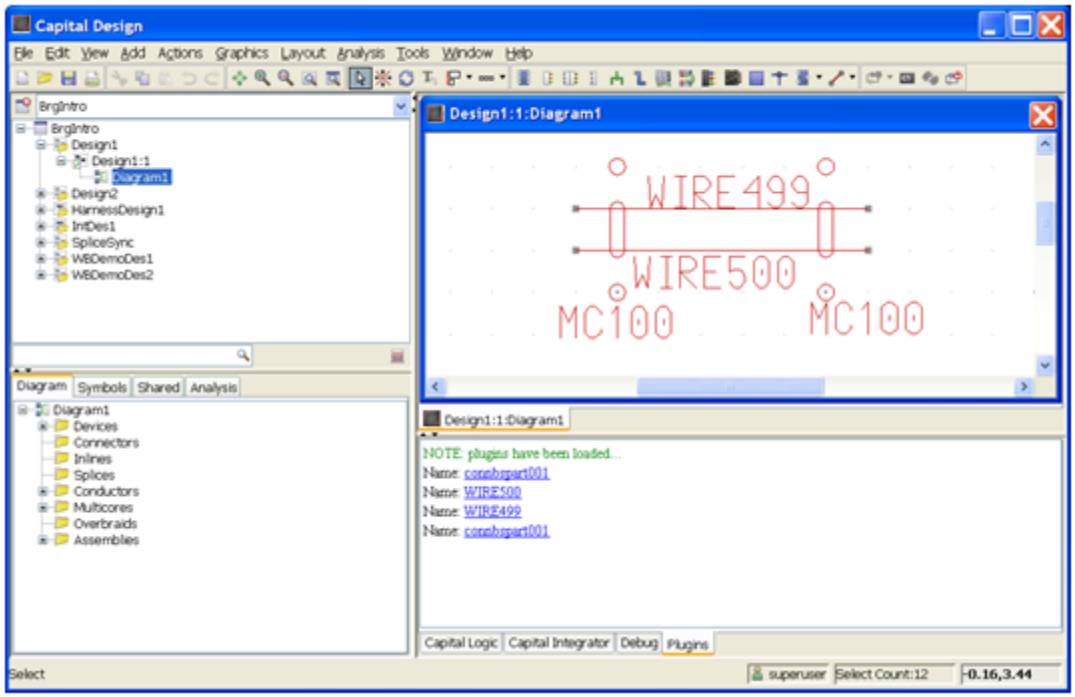
3) Now we are in a position to try out the plugin.

   a. Ensure the plugins dialog is closed.

   b. Open an existing design or create a new simple design for test purposes. Ensure there are at least 3 objects to make the selection set interesting.

   c. Select a number of objects and click on the Tools -> Custom -> Display Info menu (this is the menu that was created to trigger your new plugin).

   d. You should see something like the following displayed in the Output Window:



Note: you should also be able to access the "Display Info" menu from the context menu. The context menu is available as a pop-up menu in the diagram window. It is also available as a pop-up menu in tabs on the lower left hand side of the application.

4) However, even if no objects are selected, the Display Info menu is still present on the menu. Open the menu with no objects selected and confirm this. We need to ensure that the menu is only available when there is a selection available.

   a. Edit the `DisplayInfoAction.java` file.

   b. Edit the isAvailable() method in the file and add the following code:

```
public boolean isAvailable(IXApplicationContext applicationContext)
{
      return !applicationContext.getSelectedObjects().isEmpty();
}
```

   Note: here we are indicating that the menu should only be available if there are objects selected.

   c. Again, change the version number of the plugin, by editing the getVersion() method:

```
public String getVersion()
{
      return "0.3";
}
```

   d. Save the file.

   e. Rebuild your plugin by typing the following command in the DOS/UNIX console:

```
ant
```

   f. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check for (and fix) errors in the `DisplayInfoAction.java` file and try again.

   g. Go back to Capital Design.

   h. Click the File -> Plugins… menu to open the Plugins dialog and click the [Reload All] button to reload the plugins. You should ensure that the version number for the "Display Info" dialog has changed. Close this dialog when you have confirmed this.

   i. You should now find that the "Display Info" menu is only available when there are objects selected.

5) Now we need to display information about the attributes of the selected objects.

a. Edit the `DisplayInfoAction.java` file.

b. Edit the execute() method in the file and add the following code:

```
public boolean execute(IXApplicationContext
applicationContext)
{
      // Get the IXOutputWindow so you can log messages.
      IXOutputWindow outputWindow =
            applicationContext.getOutputWindow();

      // get the set of selected objects
      Set<IXObject> selectionSet =
            applicationContext.getSelectedObjects();
      for(IXObject selection : selectionSet)
      {
            outputWindow.println("Name: " +
selection.toHTML());
            for (IXValue value : selection.getAttributes())
            {
                  outputWindow.println(
                        "  " +
                        value.getName() +
                        " = " +
                        value.getValue());
            }

      }

      // Remember to return true unless something has gone
wrong.
      return true;
}
```

Note: the use of ` ` - these HTML tokens ensure that spaces are placed in the output window.

c. Change the version number of the plugin, by editing the getVersion() method:

```
public String getVersion()
{
      return "0.4";
}
```

d. You will also need to add the following import:

```
import com.mentor.chs.api.IXValue;
```

e. Save the file.

f. Rebuild your plugin by typing the following command in the DOS/UNIX console:

```
ant
```

g. If everything is fine, you will eventually get a "BUILD SUCCESSFUL" message. If you do not, you will need to check for (and fix) errors in the `DisplayInfoAction.java` file and try again.

h. Go back to Capital Design.

i. Click the File -> Plugins… menu to open the Plugins dialog and click the [Reload All] button to reload the plugins. You should ensure that the version number for the "Display Info" dialog has changed. Close this dialog when you have confirmed this.

j. When you click on the "Display Info" menu now (after selecting some objects) you should now see a list of attributes and their values with each of the selected objects.

6) You should continue to experiment with this plugin in the following ways:

a. See if you can change the `getTriggers()` menu to make this custom action only appear on the context menu.

b. Try modifying this custom action to display a list of properties as well as attributes.

At this point, you should know the following:

- How to iteratively develop Java Custom Actions.

Guidance

IXApplicationContextListener

To allow other plugins to have access to the `IXApplicationContext` they can implement the `IXApplicationContextListener` interface. This will force those plugins to implement the following method:

`setApplicationContext()` – This will be called by Capital to pass you the `IXApplicationContext`. You should store this in an instance variable so it can be used later.

You should not rely on having an `IXApplicationContext` set early on in the life of your plugin. When the default constructor for the custom action is called, the application context will not have been set yet. Customers should also not expect the application context to be set in the `IXPlugin` `getName()`, `getDescription()` and `getVersion()` methods either.

Displaying Dialogs

The following pattern should be used in the `execute()` method if you need to display dialogs. Note: if you are not familiar with developing dialogs in Java, you **MUST** go on a training course provided by a 3rd party first.

```
/**
 * @param applicationContext - the application context
 *
 * @return true/false.
 */
public boolean execute(IXApplicationContext applicationContext)
{
      // You should use a modal 'JDialog' rather.
      JDialog dialog = new JDialog(
            applicationContext.getParentFrame(),
            true);

      // Create your dialog using the name of the action
       // as the title of the dialog (recommended).
      dialog.setTitle(getName());

      // Construct your dialog here.
```

```
        // Display your dialog.
        dialog.pack();
        dialog.setVisible(true);

        // Remember to return true unless something has gone wrong
         // or the user has clicked [CANCEL] on your dialog.
        return true;
}
```

A template for this has been provided in the Capital installation directory in the following location:

```
doc/plugin/examples/Java/TemplateGUIAction.java
```

HTML in the Output Window

With respect to HTML tags, you can display some quite complex HTML in the output window of the application. This can be a useful alternative to displaying information in a dialog. Here is a simple example:

```
/**
 * @param applicationContext - the application context
 *
 * @return true/false.
 */
public boolean execute(IXApplicationContext applicationContext)
{
        // Get the IXOutputWindow so you can log messages.
        IXOutputWindow outputWindow
            = applicationContext.getOutputWindow();

        // Implement your action here...
         // (note the 'optional' use of HTML tags)
        outputWindow.println("<b>Hello World!</b>");

        // Remember to return true unless something has gone wrong.
        return true;
}
```

However, you need to ensure that the start and end of any HTML tags used are **both** present in a single `IXOutputWindow.println()` statement. In the example above, both the `<b>` (start tag) and the `</b>` (end tag) are present to make the text bold. This would not work if the `<b>` (start tag) was in one `println()` and the `</b>` (end tag) was in another. This is also true of more complex HTML tags like lists and tables. The entire list `<ol>` (start tag) and `</ol>` (end tag) and any internal tags used must be in a single `println()`. In such situations, you can use a

`StringBuilder` object to construct the complex HTML before passing it to a single `println()` method call.

For formatting, if you want to ensure that a space is displayed in the output window you can use the non-breakable space ` ` HTML token.

With respect to hyperlinks, `IXObjects` have a `toHTML()` method to help with writing to the Output Window. This method will return a string that contains a hyperlink that can be used in a `println()` method call. When you click on this link in the output window it will select the object in the diagram if possible. If the object itself was a diagram, it will try to open the diagram. If the design was not loaded, it will try to load the design the object or diagram is in. The name of the object will be inserted into the hyperlink if no parameter is passed in. If you require a different message in the hyperlink then you can use an overloaded version of the `toHTML()` method that takes a message as a parameter.

Hyperlinks to web pages (intranet or internet) can also be written to the output window. This will cause an external HTML browser to be launched. However you must configure the browser used based on the platform you are running on. By default it is setup for the Windows platform for Internet Explorer, so you should see the following in `clientprops.xml`:

```
<externalhyperlinks
     enabled="true"
     handler="externalProgram"
     commandLine="explorer" />
```

You can switch this to use FireFox on Windows by doing the following (in this example FireFox is installed in **C:\Program Files\Mozilla Firefox**):

```
<externalhyperlinks
      enabled="true"
      handler="externalProgram"
      commandLine="C:\Program Files\Mozilla Firefox\firefox.exe" />
```

Once this has been configured, to insert a link to the Mentor Graphics web site (for example) you can do the following:

```
context.getOutputWindow().println(getName(),
      "Please click <A href=\"http://www.mentor.com\">here</A>");
```

Changing Properties and Attributes

To ensure that the integrity of your data, there are a number of rules that the plugin framework enforces:

1) You are currently only allowed to modify properties/attributes within a Custom Action

2) You are only allowed to modify properties/attributes if the `IXAction.isReadOnly()` method returns false. This ensures no attempt is made to modify an object in a design locked by someone else (opened read-only).

3) You are only allowed to modify properties/attributes within the `IXAction.execute()` method.

4) You are only allowed to modify properties/attributes on objects contained within the current design that has focus if your action is an `IXLogicAction`, `IXTopologyAction`, `IXIntegratorAction` or `IXHarnessAction`. Also note that the changes done to the attributes and properties on all objects are undoable except for those made on the shared objects in Capital Logic.

5) You can modify properties/attributes on objects across all the designs of the current project only if your action is an `IXApplicationAction`. However, the changes done to the attributes and properties are not undoable.

Also to ensure data integrity, the changes that you want to be applied are first stored in an `IXAttributeSetter` object (you do not modify the objects directly). For each object whose property/attributes need modification, one of these objects needs to be obtained. To obtain such an object, call the `IXWriteableObject.getAttributeSetter()` method on the object. If any of the above rules are broken, then the `IXWriteableObject.getAttributeSetter()` method will return null (so it is very important that you check that null has not be returned). Here is an example:

```
boolean execute (IXApplicationContext context)
{

      //…
      IXObject xObject = …
      // …

      // Do I have an object that is writable?
      if( xObject instance of IXWriteableObject )
      {
```

```
                IXWriteableObject writeableObject
                        = (IXWriteableObject) xObject;
                IXAttributeSetter result
                        = writeableObject.getAttributeSetter();
                // Are we in READ/WRITE mode?
                if(result != null)
                {
                      result.addProperty("xxx", "100");
                }
        }
        return true;
}
```

You can use `IXAttributeSetter` to modify the value of attributes and add, modify or remove properties.

However, again to ensure data integrity, these changes are not applied immediately. They will applied after the `execute()` method returns and ONLY if it returns true. If your `execute()` method returns false, any changes stored in the `IXAttributeSetter` objects are discarded.

Again, ensure data integrity, the values are checked to ensure they are correct for the type of attribute/property. If there are any problems, then a message will be displayed indicating why the change was not accepted.

Examples

At this point, you should have a good idea how to implement this type of plugin. Some examples have been provided to help you. These examples can be found in the following location within the Capital installation:

```
doc/plugin/examples/Java/src/com/example/plugin/action
```

The following examples are good starting points:

- `DeviceCounterIncrementAction.java` – this example shows how you modify properties.
- `PropertyHtmlTableAction.java` – this example shows how you display complex HTML in the output window.
- `PropertyViewerTreeAction.java` – this example shows how Java GUIs are displayed.

You can copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).